Lab 1 Report

Proud Dumrongpun

Component A Deliverables

1. **Interview notes**

Answer:

Dorothy Interview

Current Process

- Orders handled via Amazon / Non-Amazon
- Workflow: Request → Approved → Purchase → Delivered → Confirm
- Volume: 20–50 orders/month
- Organized by cohort / by project
- Each team has its own budget

Pain Points

- Received documents are physical, taking 3 days / 2 ways
- Ambiguous forms lead to incorrect data entry
- Current process is not easy to submit

Desired Improvements

- Submission should be made easier
- Existing report showing how much each team purchased
- Front desk received receipt
- Checking email → packing slip step
- Order number → send refund → refunded to team budget
- If rejected → put back order (ask before cancel)
- Prioritized & easy submission process

Required Form Fields

- Team name
- CFO
- Provider / Supplier
- Quantity
- Name of items
- Price
- Purchase link
- Note
- Approval from Instruct
- Amazon / Non-Amazon toggle

Additional Features Requested

- Export CSV
- Supplier dropdown (including non-Amazon options)
- Approval view by 1pm / week
- Cancel purchase option

2. **Problem statement: 1 sentence in the template**

Answer:

When a student needs to submit a purchase request, they currently fill in a Google Spreadsheet with no input validation, which causes incorrect or incomplete data entry and makes it hard to track team budget spending.

<u>Component B Deliverables</u>

1. **AI Usage Log: Document at least 3 interactions with AI tools (include Cursor prompts from build/iteration). For each, write:**
    - **the prompt you gave**
    - **what the AI produced**
    - **whether it worked on the first try, and what you fixed if it did not**

Answer: I used **two AI tools** — first **Claude** for designing my development work, and **Cursor to** code.

Claude;

1. I used Claude to plan the development steps.

My prompt:

I'm going to create a web application for purchasing. I've completed the workflow. This app needs to handle these three workflows and must have authorization access, an approval workflow, and a messaging function so that the Requester, Approver, and Purchasing Team can communicate within the application. What steps should I follow when prompting Cursor to build this?

what the AI produced:

It gave me 6 Steps

No.	Build
1	Database + Models
2	Authentication

No.	Build
3	PR Workflow
4	Receiving Workflow
5	Returning Workflow
6	Messaging
7	Dashboard + UI

whether it worked on the first try, and what you fixed if it did not

Yes, it worked. So I know the steps in order to develop the app.

2. I used Claude to help me prepared detailed prompts for Cursor

My prompt:

For Step 1: Database + Tech Stack, Help me prepare prompt for Cursor.

The detail are as below.

- Using Streamlit
- Creating database tables for User, Purchase request,purchase order, inventory_receive, return_notes, budget, messages

what the AI produced

Create a Python Streamlit web application for a Purchasing Management System. Tech Stack: - Frontend: Streamlit - Database: SQLite with SQLAlchemy - Authentication: streamlit-authenticator Create the following database tables: - users (id, name, email, password, role) roles: 'requester', 'approver', 'head_of_purchasing', 'purchasing_team' - purchase_requests (id, pr_number, requester_id, status, budget_amount, description, created_at) status: 'draft', 'submitted', 'approved', 'rejected' - purchase_orders (id, po_number, pr_id, purchasing_team_id, status, created_at) status: 'open', 'success', 'rejected' - inventory_receive (id, ir_number, po_id, status, created_at) status: 'open', 'accepted', 'returning' - return_notes (id, rn_number, requester_id, ir_id, status, reason, created_at) status: 'draft', 'submitted', 'approved', 'rejected', 'closed' - budget (id, total_budget, consumed_budget, returned_budget) - messages (id, reference_id, reference_type, sender_id, message, timestamp) Also insert dummy data: - 1 user per role (4 users total) - 2-3 sample purchase requests - 1 sample budget record: total=1,000,000

whether it worked on the first try, and what you fixed if it did not

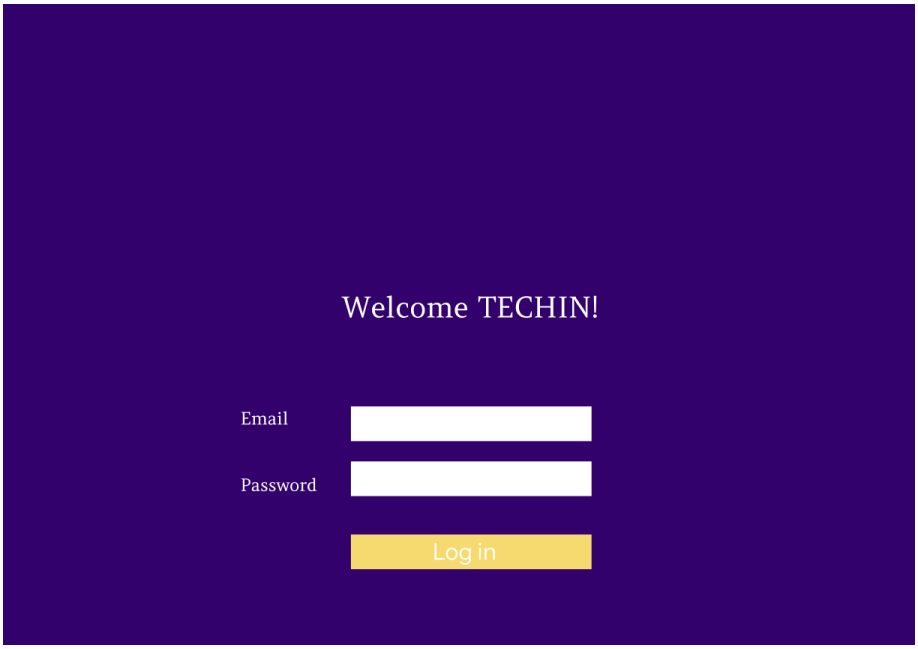
No, I reviewed the table details and found that some table were still missing. So I told Claude to include the following tables:

1. Authorization Role to determine which roles can view or modify fields in the Purchase Request.
2. Class and Project to manage budgets and populate dropdown lists.
3. Supplier to populate dropdown lists.
4. Validation Rules to validate fields, for example, Quantity and Price must be numbers.

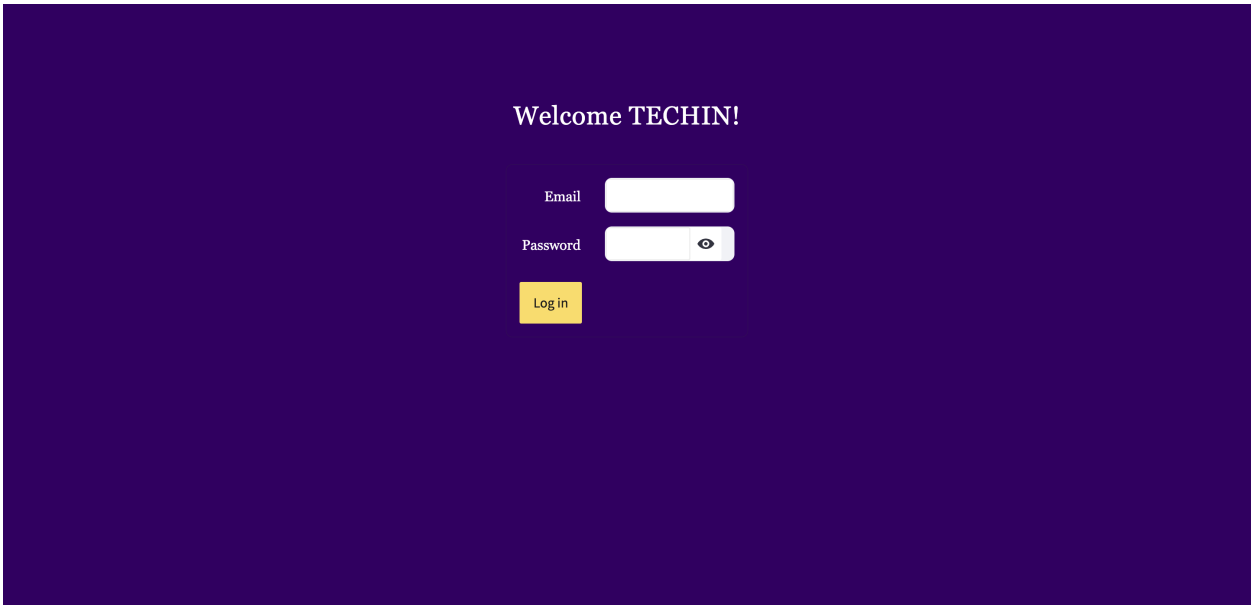
3. I used Cursor to develop based on the prompt I got from Claude and UI from Figma

the prompt you gave

Design log in page as attached photo



what the AI produced



whether it worked on the first try, and what you fixed if it did not

The function was fine but the layout and portion were not precise. But I didn't fix it since it was a minor issue.

2. **Accessibility baseline results: Briefly report:**

Answer:

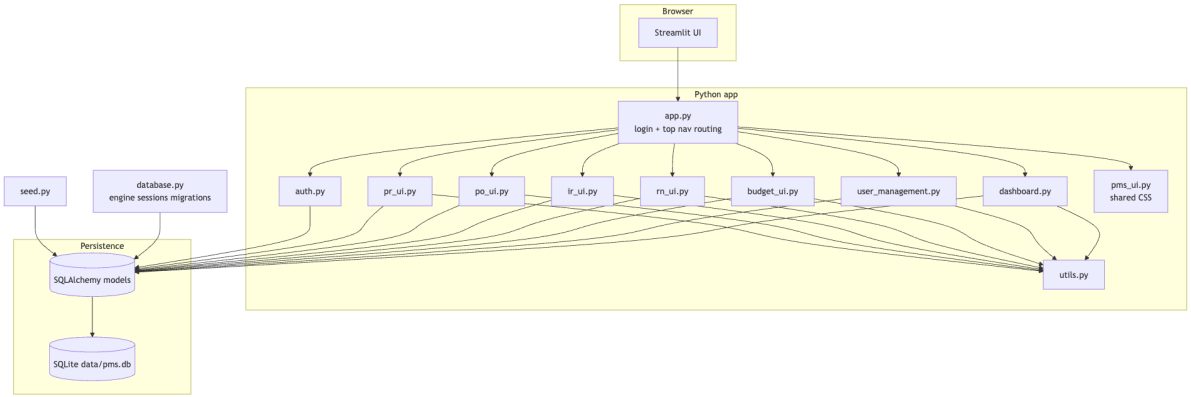
Check	Result	Notes / fix
Color contrast	Pass (after fix)	Login submit used white text on #F7DC6F → likely fail vs WCAG AA. Fix applied: app.py login form CSS now uses color: #1a1a1a on the yellow button (normal-weight text). Re-check primary green buttons (#16a34a on white) with a contrast tool; they generally pass for white label text.
Semantic headings	Pass (with caveat)	Logged-in areas use Streamlit st.title / st.subheader (e.g. Dashboard, Budget management, workspace sections)—these map to heading semantics in the DOM. Caveat: Login welcome text is a styled <p> (.login-welcome), not an <h1>; acceptable for a single screen, but add st.title hidden for SRs only if your auditor requires a literal <h1> on every route.

Tools: browser DevTools Accessibility tree, axe/WAVE, or WebAIM Contrast Checker.

<u>Component C Deliverables</u>

1. **Architecture diagram**

Answer:



2. Design Decision Log

Answer:

Design decision log

Decision	Choice	Rationale
UI framework	Streamlit	Fast lab UIs, forms/tables without a separate front-end build.
Database	SQLite file under data/	Zero server setup; portable for grading and demos.
ORM	SQLAlchemy 2.x + declarative models	Typed models, migrations via database.py, relationships for PR→lines→PO→IR.
Auth	Session state + bcrypt hashes in users	Matches lab scope; not OIDC/production SSO.
Module split	One workspace file per document type (pr_ui, po_ui, …)	Easier navigation and parallel work than a single giant file.
Permissions	Permission + MenuVisibility + role flags	Supports row-level "own only" and field-level edit rules for PRs.
Document numbers	DocumentNumbering table + utils.next_document_number	Central place for year rollover and padded sequences.
Budget	BudgetTransaction consume/return + class/team caps	Mirrors procurement reserving and releasing funds on status changes.

Component D Deliverables

1. Validation results (the completed smoke-test table)

Answer:

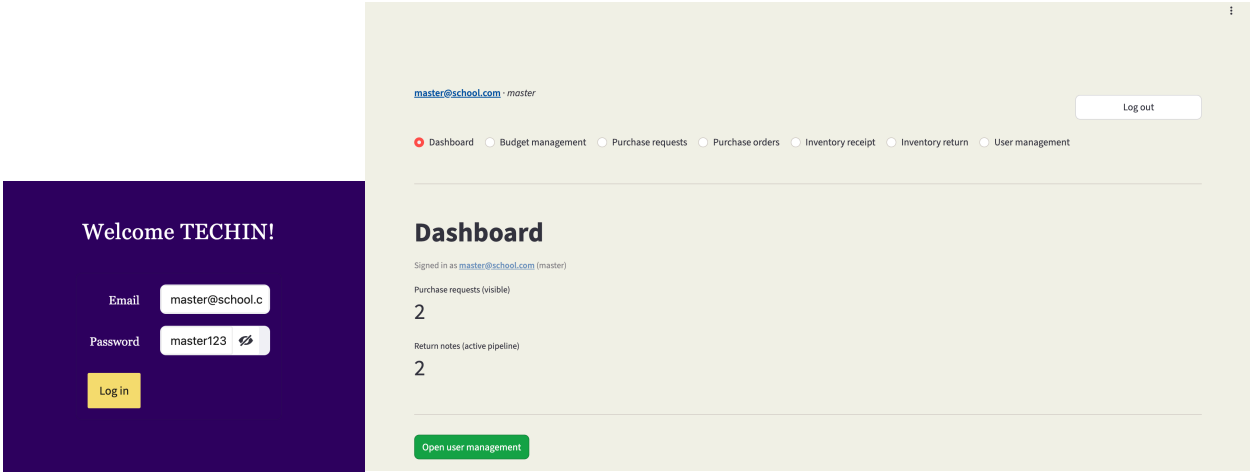
#	Test Case	Steps	Expected	Pass?
1	Test successful Login	Login as master by inputting correct password	Lands on app with top navigation	Y
2	Test Invalid login	Login as master by inputting wrong password	"Invalid credentials."	Y
3	Test Purchase Request Creation	Log in as Requester to create Purcase Request and Submitt it	Status change to Submitted. The status log must record the change of documents	Y
4	Test Description Validation	Don't input Description and Submit it	Show Error message and can't submitt PR	Y
5	Test 1st Approver	Logged in as approver and approve PR by approve 1 line item and Reject 1 line item.	Status change from Submitted to Review	Y
6	Test 2nd Approver	Logged in as Head of Purchasing and approve PR	Status change from Reviewd to Approve	Y
7	Test PO creation	Logged in as Purchasing Team to create PO	PO is created	Y
8	Test IR Creation	Logged in as Purchasing Team to create IR	IR is created	Y

9	Test Document Check list on IR	Logged in as Purchasing Team to check all the document Receive : Invoice,Delivery note and PO and then click button Ready to pick up	IR status change from open to Ready_for_pick up	Y
10	Test User inventory acceptance	Logged in as Reqester to accept Inventory	IR status change from open to Close	Y
11	Test Return Note Creation	Logged in as Reqester to Create Return Note and Submit it	Return Note is created and Submitted	Y

2. Screenshot of Testing

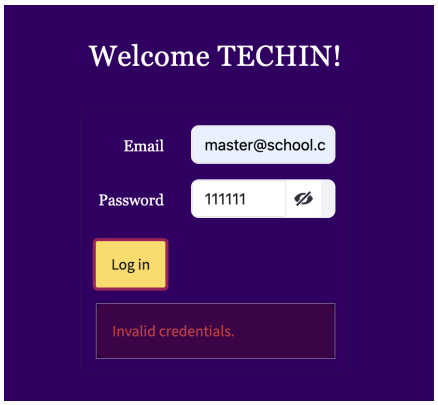
Answer:

Test Case#1 Test successful Login

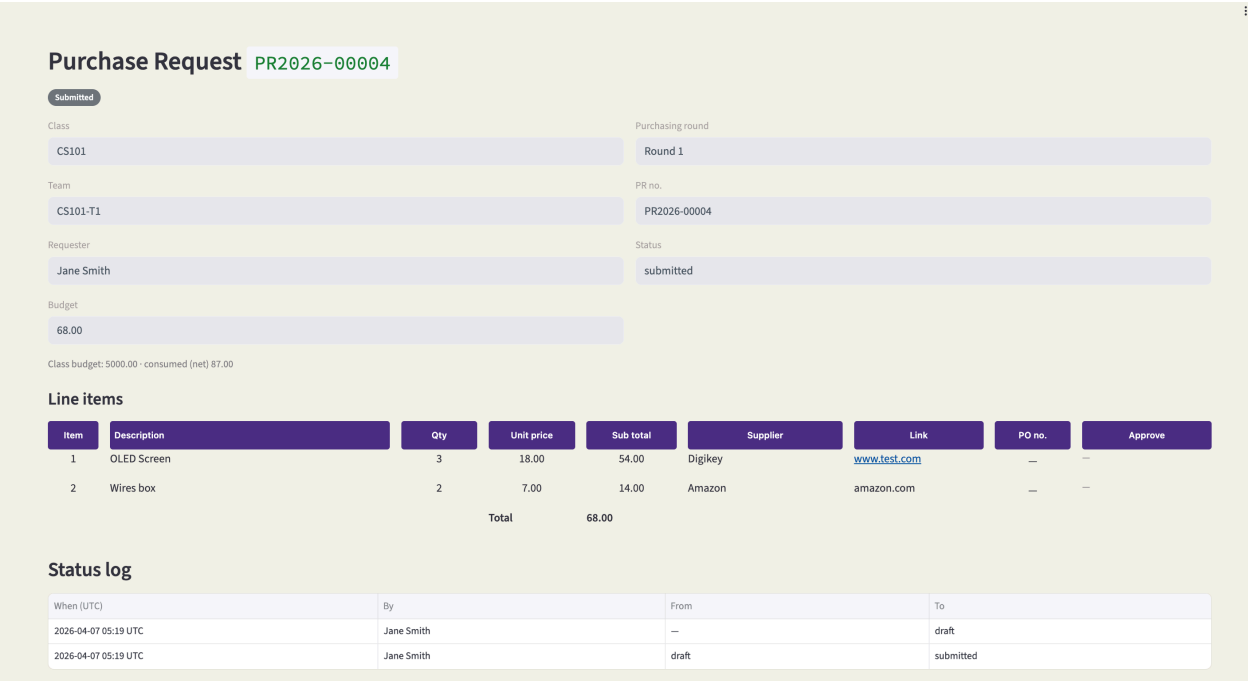


Success fully logged in as Master

Test Case#2 Invalid Login



Test Case#3 Success fully create PR and submitted it



Purchase Request PR2026-00004

Submitted

Class: CS101

Purchasing round: Round 1

Team: CS101-T1

PR no.: PR2026-00004

Requester: Jane Smith

Status: submitted

Budget: 68.00

Class budget: 5000.00 · consumed (net) 87.00

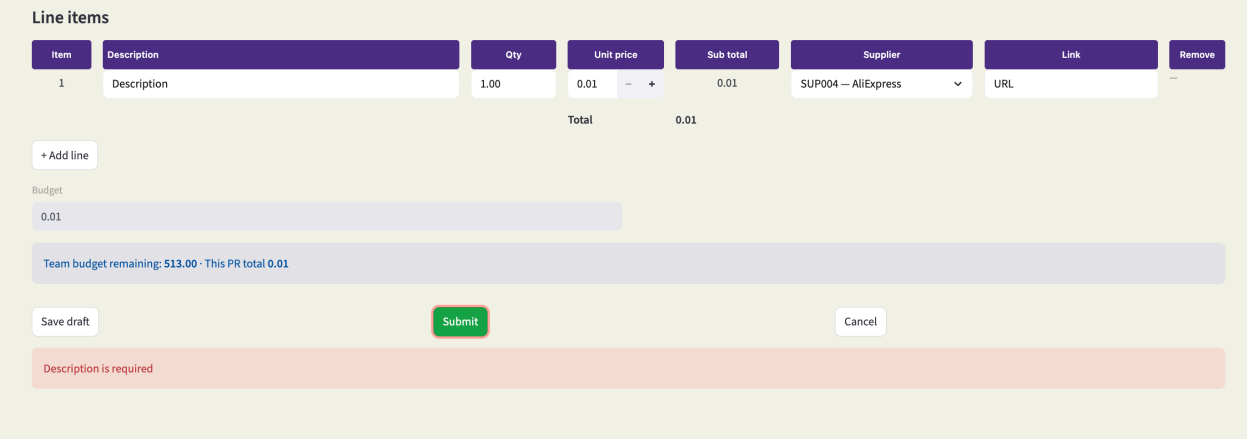
Line items

Item	Description	Qty	Unit price	Sub total	Supplier	Link	PO no.	Approve
1	OLED Screen	3	18.00	54.00	Digikey	www.test.com	—	—
2	Wires box	2	7.00	14.00	Amazon	amazon.com	—	—
			Total	68.00				

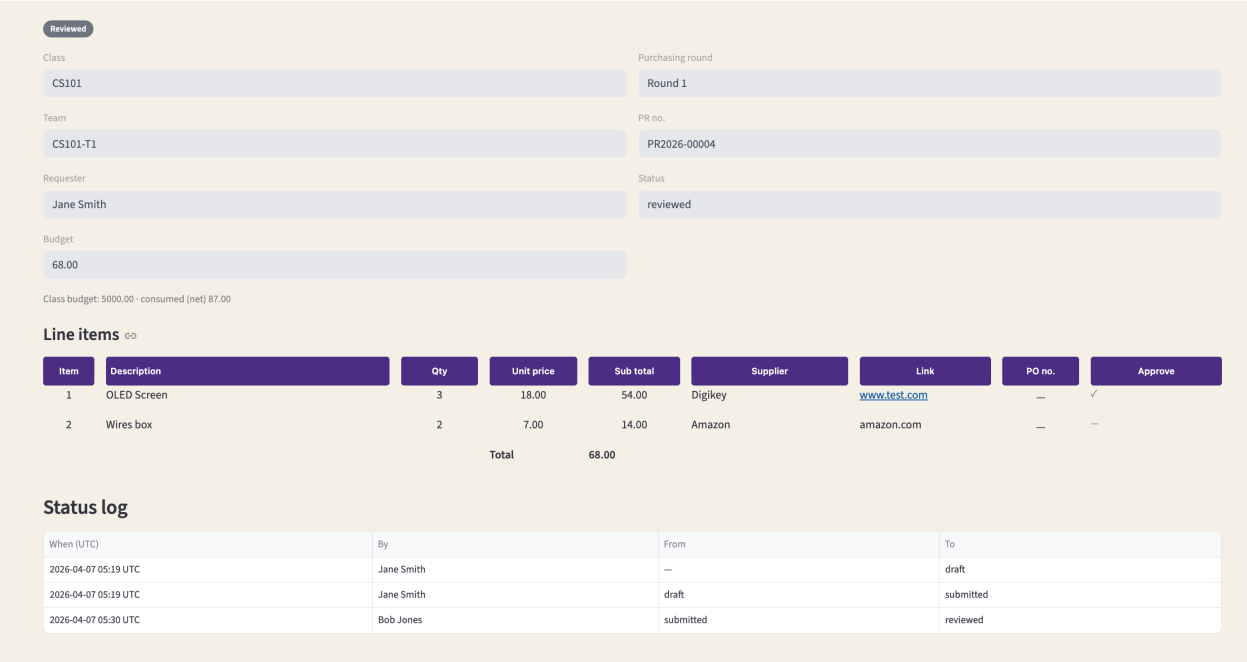
Status log

When (UTC)	By	From	To
2026-04-07 05:19 UTC	Jane Smith	—	draft
2026-04-07 05:19 UTC	Jane Smith	draft	submitted

Test Case#4 Test Description Validation



Test Case#5 Test 1st Approver



Test Case#6 Test 2nd Approver

Team

CS101-T1

PR no.

PR2026-00004

Requester

Jane Smith

Status

reviewed

Budget

68.00

Class budget: 5000.00 · consumed (net) 87.00

Line items

Item	Description	Qty	Unit price	Sub total	Supplier	Link	PO no.	Approve
1	OLED Screen	3	18.00	54.00	Digikey	www.test.com	—	✓
2	Wires box	2	7.00	14.00	Amazon	amazon.com	—	✗
			Total	68.00				

Status log

When (UTC)	By	From	To
2026-04-07 05:19 UTC	Jane Smith	—	draft
2026-04-07 05:19 UTC	Jane Smith	draft	submitted
2026-04-07 05:30 UTC	Bob Jones	submitted	reviewed
2026-04-07 05:36 UTC	Alice Brown	reviewed	approved

Messages

No messages yet.

Test Case#7 Create PO

Line items

Item	Description	Qty	Unit price	Sub total	Supplier	Link	PO no.	Create PO	Cancel PO	Approve
1	OLED Screen	3	18.00	54.00	Digikey	www.test.com	—	Create PO	—	✓
2	Wires box	2	7.00	14.00	Amazon	amazon.com	—	—	—	✗
			Total	68.00						

Status log

When (UTC)	By	From	To
2026-04-07 05:19 UTC	Jane Smith	—	draft
2026-04-07 05:19 UTC	Jane Smith	draft	submitted
2026-04-07 05:30 UTC	Bob Jones	submitted	reviewed
2026-04-07 05:36 UTC	Alice Brown	reviewed	approved

PO number is generated on PR

Line items

Item	Description	Qty	Unit price	Sub total	Supplier	Link	PO no.	Create PO	Cancel PO	Approve
1	OLED Screen	3	18.00	54.00	Digikey	www.test.com	PO2026-00002	—	Cancel PO	✓
2	Wires box	2	7.00	14.00	Amazon	amazon.com		—	—	✗
			Total	68.00						

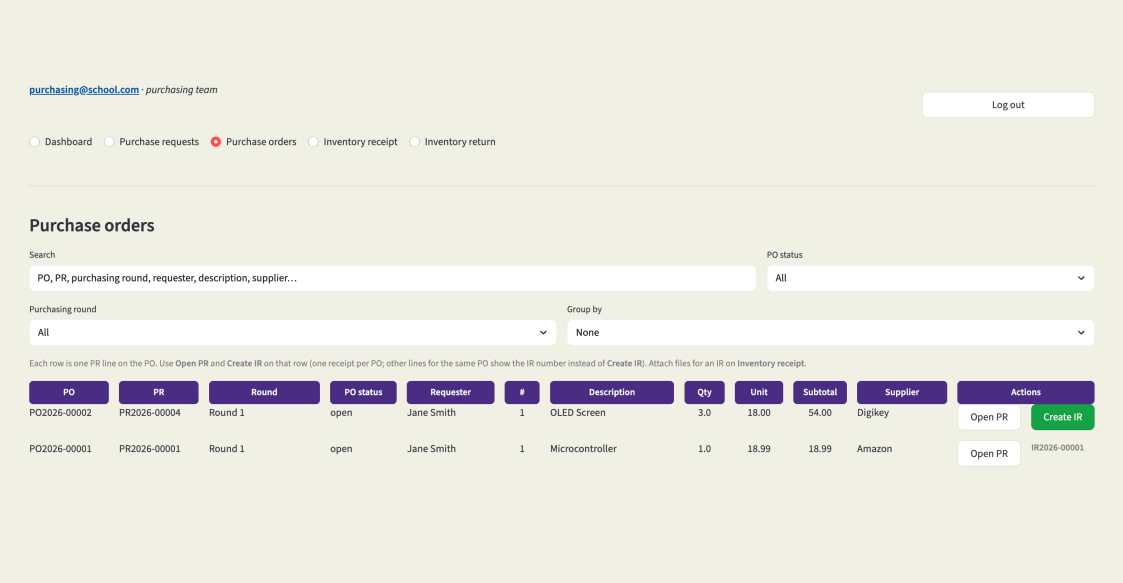
Status log

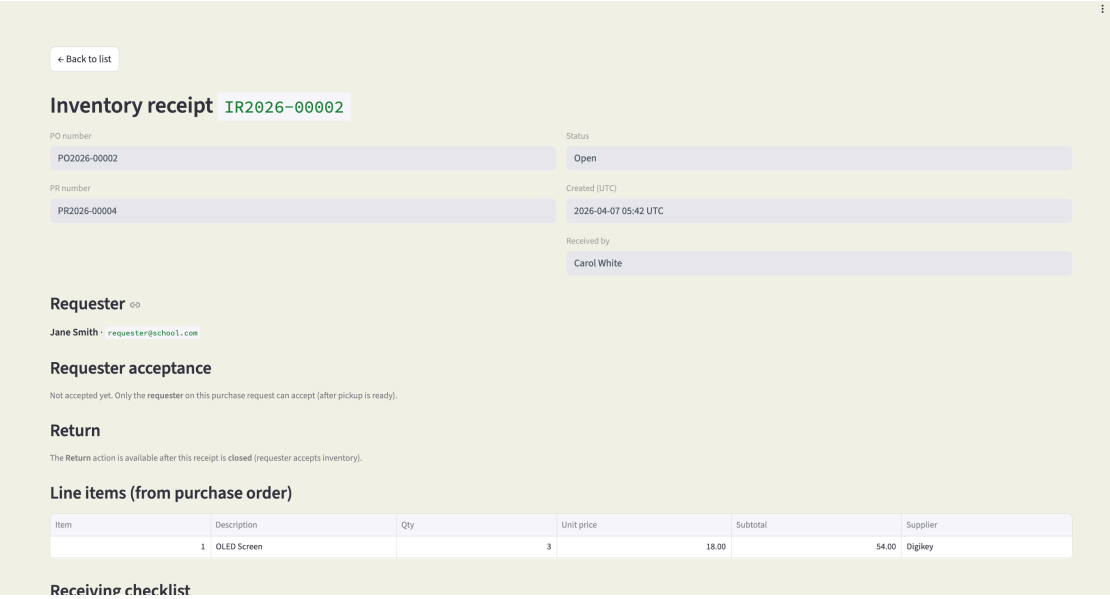
When (UTC)	By	From	To
2026-04-07 05:19 UTC	Jane Smith	—	draft
2026-04-07 05:19 UTC	Jane Smith	draft	submitted
2026-04-07 05:30 UTC	Bob Jones	submitted	reviewed
2026-04-07 05:36 UTC	Alice Brown	reviewed	approved

Messages

Test Case#8 Create Inventory Receipt

On Purchase order page, Purchasing Team will see Create IR button.





Test Case#9 Test Document Check list on IR

Receiving checklist

Tick each box when verified, then click **Save checklist**.

- [x] PO document OK
- [x] Delivery note OK
- [x] Invoice OK

Save checklist

Ready for pickup

All checklist items are complete. Notify the requester that items are ready to collect.

Ready to pick up

Status log

When (UTC)	By	From	To
2026-04-07 05:42 UTC	Carol White	—	open

This receipt is **ready for pickup**; the checklist can no longer be edited.

Ready for pickup

This receipt is **Ready to pick up**. The requester can accept once they have collected the items.

Status log

When (UTC)	By	From	To
2026-04-07 05:42 UTC	Carol White	—	open
2026-04-07 05:46 UTC	Carol White	open	ready_for_pickup

Attachments

Download existing files or upload a new one (max 15 MB per file).

No files attached yet.

Add attachment

Upload 200MB per file

Test Case#10 Test Requester Inventory Accepttion

Inventory receipt IR2026-00002

PO number: PO2026-00002

Status: Ready to pick up

PR number: PR2026-00004

Created (UTC): 2026-04-07 05:42 UTC

Received by: Carol White

Requester

Jane Smith · requester@school.com

Requester acceptance

Purchasing marked this receipt **ready to pick up**. Confirm after you collect the items.

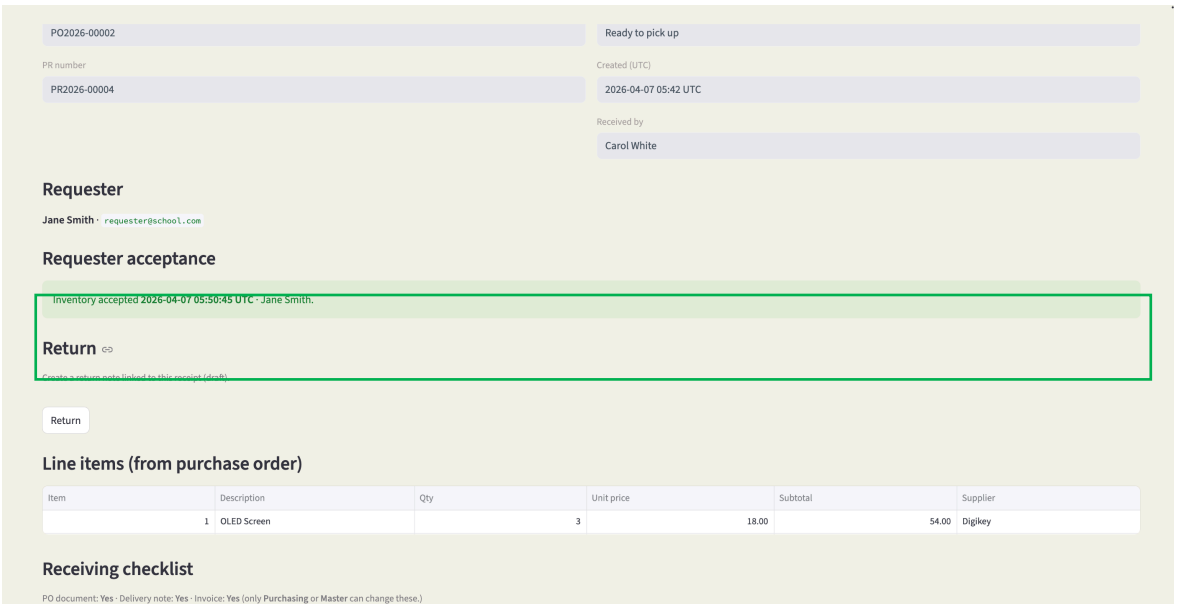
Accept inventory

Return

The **Return** action is available after this receipt is **closed** (requester accepts inventory).

Line items (from purchase order)

Item	Description	Qty	Unit price	Subtotal	Supplier
1	OLED Screen	3	18.00	54.00	Digikey



PO2026-00002

Ready to pick up

PR number

PR2026-00004

Created (UTC)

2026-04-07 05:42 UTC

Received by

Carol White

Requester

Jane Smith · `requester@school.com`

Requester acceptance

Inventory accepted **2026-04-07 05:50:45 UTC** · Jane Smith.

Return

Create a return note linked to this receipt (draft).

Return

Line items (from purchase order)

Item	Description	Qty	Unit price	Subtotal	Supplier
1	OLED Screen	3	18.00	54.00	Digikey

Receiving checklist

PO document: **Yes** · Delivery note: **Yes** · Invoice: **Yes** (only **Purchasing** or **Master** can change these.)

Test Case#11 Return Note Creation

Return note RN2026-00004

Status: **Submitted** · Receipt **IR2026-00002** · PO **PO2026-00002** · PR **PR2026-00004**

Requester

Jane Smith

Items (from purchase order / receipt)

Item	Description	Qty	Unit price	Subtotal	Supplier
1	OLED Screen	3	18.00	54.00	Digikey

Return reason

It's broken

Actions

Awaiting **Head of Purchasing** (or **Master**) approval.

Status log

When (UTC)	By	From	To
2026-04-07 05:51 UTC	Jane Smith	—	draft
2026-04-07 05:52 UTC	Jane Smith	draft	submitted

Test Case#12 Test Return Note Approval

Return note RN2026-00004

Status: **Closed** · Receipt **IR2026-00002** · PO **PO2026-00002** · PR **PR2026-00004**

Requester

Jane Smith

Items (from purchase order / receipt)

Item	Description	Qty	Unit price	Subtotal	Supplier
1	OLED Screen	3	18.00	54.00	Digikey

Return reason

It's broken

Actions

This return note is **completed** and closed.

Status log

When (UTC)	By	From	To
2026-04-07 05:51 UTC	Jane Smith	—	draft
2026-04-07 05:52 UTC	Jane Smith	draft	submitted
2026-04-07 05:54 UTC	Alice Brown	submitted	approved
2026-04-07 05:54 UTC	Alice Brown	approved	closed

Quality gate checklist
Answer:

- pip install -r requirements.txt succeeds on a clean venv
- streamlit run app.py starts with no import errors
- At least one demo login works as documented in README.md
- Smoke-test table above completed (all checked or failures noted)
- No committed secrets (passwords in README are demo-only; .env not required)
- data/ remains gitignored; DB is reproducible via first-run seed
- Accessibility spot-checks documented in Component B

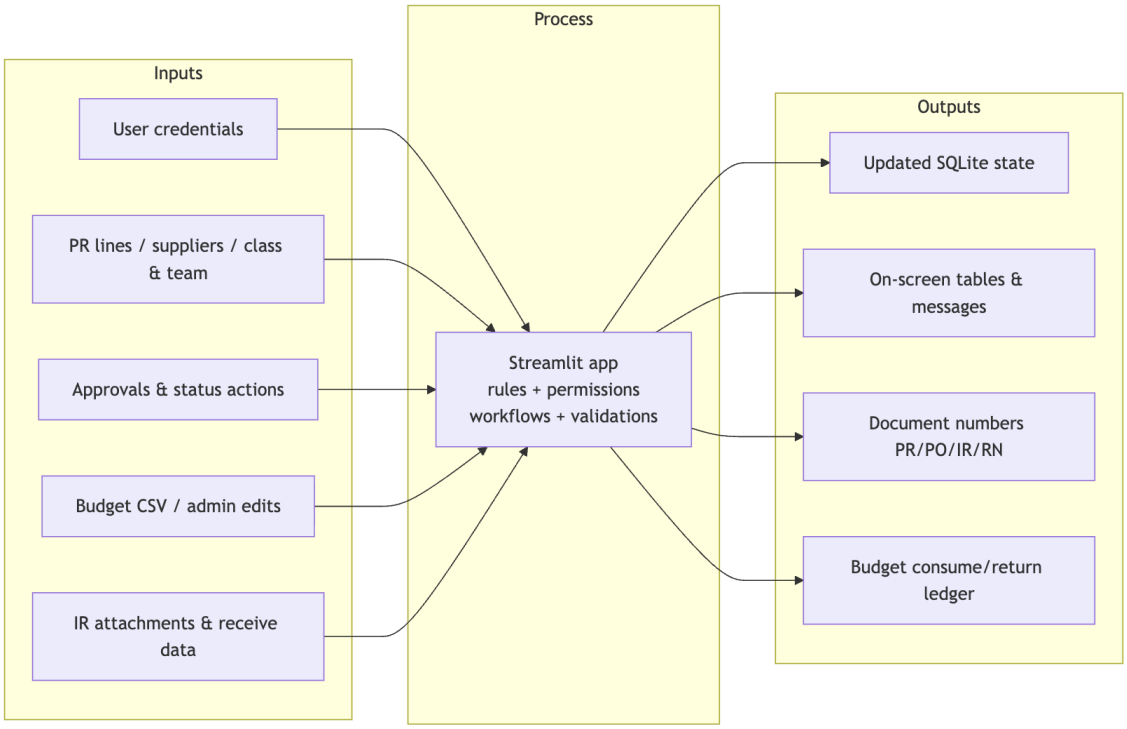
Component E Deliverables

Implementation in this repo

Area	Location
Entry + routing + login shell	app.py
Data model	models.py
DB engine, migrations, helpers	database.py
Initial data	seed.py
Auth	auth.py
Workspaces	pr_ui.py, po_ui.py, ir_ui.py, rn_ui.py, budget_ui.py, dashboard.py, user_management.py
Shared styling	pms_ui.py
Validation / numbering / budget math	utils.py

Data layout: SQLite database data/pms.db with tables for roles, users, students, classes, teams, suppliers, purchase requests (+ line items, messages, status history), purchase orders, inventory receipts (+ attachments/history), return notes, budget transactions, document numbering, permissions, and menu visibility.

I / P / O diagram



Edge-case validation notes (2)

1. **PR total vs team budget cap**
   - Behavior: Saving/submitting can be blocked when the PR total exceeds remaining team budget (team_budget_cap_remaining, errors in pr_ui.py).
   - Why it matters: Prevents overspending against allocations; mirrors how student purchasing is capped per team/class in the interview scenario.
2. **Invalid credentials**
   - Behavior: attempt_login returns no user; UI shows “Invalid credentials.”
   - Why it matters: Avoids leaking whether an email exists; keeps failed auth obvious without crashing.

Data integrity assert (in code)

Location: seed.py, at the end of _seed_data, after all roles are inserted.
Purpose: After seeding, the database must contain exactly five roles (master, requester, approver, head_of_purchasing, purchasing_team). If that invariant breaks, seed logic regressed.

```
seeded_roles = session.query(Role).count()
assert seeded_roles == 5, f"Expected 5 seeded roles, got {seeded_roles}"
```

Prompt log excerpt:

Stage	Prompt (excerpt)	What changed and why
Initial	Build a role-based purchasing web app with PR → approval → PO → receiving → returns, using Python and a file database.	Set overall scope and stack.
Refinement	Use team-level budget caps on submit, not just class budget, and show remaining budget next to the PR total.	First version aligned totals to the wrong budget dimension; refinement matched Dorothy-style rules (teams compete for finite allocations).

Reflection Deliverables

Before you leave, take a few minutes to write 3-5 sentences addressing the following questions. Submit these as part of your Component B deliverables.

1. **What surprised you about AI-assisted coding? Was it easier or harder than you expected? What moment made you think "wow" or "wait, that is not right"?**

Answer: The surprise is how smart they are likeafter I just gave them the workflow and asked them what tables I should have, they were able to generate almost every required table with the details. Normally, if we did it ourselves, it would take a very long time to list everything. Some thing that is hard when I don't know the technicle that much, So when something is wrong, it took time to debug or I kept telling it to fix but I don't know exactly where part of the code need to fixed.

2. **What did the AI get wrong? Did you encounter errors, unexpected behavior, or code that did not match what you asked for? How did you fix it?**

Answer: Yes, when I use Cursor, if I don't tell it precisely what I want, it tends to generate something far from what I want and I end up having to debug it, which costs me more time compared to if I had just given it detailed instructions from the start.

3. **Could you explain your code? Pick one section of your app -- could you walk a classmate through what it does, line by line, without looking at Cursor's explanation?**

Answer:

The code:

```
def attempt_login(session: Session, email: str, password: str) -> Optional[AppUser]:
    email = (email or "").strip().lower()
```

```
if not email or not password:
    return None
user = (
    session.query(AppUser)
    .options(joinedload(AppUser.role), joinedload(AppUser.student))
    .filter(AppUser.email == email)
    .first()
)
if not user or not user.is_active:
    return None
if not verify_password(password, user.password):
    return None
return user
```

Explanation:

This function answers is to veriy username as password. It takes 3 inputs: -1.session : connection to the database -2.email : what the user typed 3.password : what the user typed and then It wil check 4 steps

1.Is anything actually typed? If email or password is empty it will reject immediately

2.Does this email exist in the database? Goes into the database and looks for a user with that email If not found, it will reject

3.Is this account active? The account might exist but be suspended or disabled If inactive, it will reject.

4.Check 4 Is the password correct? Compares the typed password against the hashed password in the database If wrong, it will reject .

4. **What did you learn from the interview? How did talking to Dorothy (Program Coordinator, Student Purchasing) change what you built, compared to what you might have built without that conversation?**

Answer: Yes, if I hadn't interviewed her, I might not have come up with the three workflow processes: Create PR, Receive Inventory, and Return. I might only have thought of the Create and Approve steps. I also might not have considered the validation of required fields or dropdown lists to reduce human input errors.